



Software Engineering Department

Braude College of Engineering

Final Project – Phase A (61998)

Evolution of Knowledge Areas in Publications

CODE: 26-1-R-16

By:

Yamit Moreno	207476102	yamitmoreno1@gmail.com
--------------	-----------	------------------------

Shaked Tadessa	318848793	shakedt67509@gmail.com
----------------	-----------	------------------------

Advisors:

Dr. Dvora Toledano-Kitai

Prof. Zeev Volkovich

Link to GitHub:

https://github.com/Yamit-Moreno/FinalProject_EvolvingKnowledgeInPublications

Abstract	1
1. Introduction	1
2. Problem Formulation	2
3. Research Fields and Academic Context	2
3.1 Citation Networks – Definitions and Structure	3
3.2 Embeddings and Clustering	3
3.2.1. Embedding Methods – General and Modern Approaches	4
3.2.2 GraphSAGE Algorithm	4
3.2.3 K-means Algorithm	5
3.2.4 Silhouette Score	6
3.2.5 Jaccard Similarity	7
3.2.6 Hungarian Method	7
3.2.7 Visualization – Alluvial Diagrams	8
4. Requirements	8
4.1 Functional Requirements	8
4.2 Non-Functional Requirements	9
5. Model Description	10
5.1 Graph Input and Dynamic Network Representation	10
5.2 Node Representation via Graph Embeddings	10
5.3 Community Detection within Snapshots	11
5.4 Robustness to Random Initialization	11
5.5 Temporal Alignment of Communities	11
5.6 Visualization of Community Evolution	11
5.7 Pseudocode	12
5.8 Model Flowchart	14
6. Statistical Analysis Methods and Implementation Technologies	14
6.1 Statistical Methods for Analyzing Model Results	15
6.1.1 Cluster Quality Evaluation	15
6.1.2 Robustness to Random Initialization	15
6.1.3 Structural Change Analysis	15
6.2 Implementation Technologies (Phase B)	15
6.2.1 Programming Language and Execution Environment	16
6.2.2 Core Libraries and Frameworks	16
6.2.3 Visualization and Result Interpretation	16
6.3 Anticipated Technical Challenges	16
6.3.1 Scalability and Dynamic Graph Processing	17
6.3.2 Identification of a Suitable Dataset	17
6.3.3 Complexity of Visualization	17
7. Evaluation and Testing Plan	17
7.1 Evaluation	17
7.2 Testing Plan	19
Table 4. Verification / Testing Matrix	19
8. AI Tools and Prompts Used	20
9. References	21
10. Appendices	22
10.1 Appendix A – Full Algorithm Pseudocode	22
10.2 Appendix B – Visualizations	25

Abstract

This project proposes a framework for analyzing and mapping structural changes in large-scale dynamic networks, using academic citation networks as a case study. The framework aims to identify patterns, trends, and transformations within research communities over time, while distinguishing significant structural changes from statistical noise inherent in complex networks.

The proposed approach integrates GraphSAGE-based node embeddings with K-Means clustering to group nodes with similar characteristics, and evaluates cluster quality using the Silhouette Score. Temporal cluster dynamics, including merges, splits, and the emergence of new research domains, are visualized using Alluvial diagrams, providing an intuitive representation of community evolution.

Overall, The framework supports robust and meaningful identification of evolving network structures, effectively handling noise and enabling dynamic network analysis. Its main contribution lies in offering a clear and structured view of network evolution, with potential applicability to other domains such as social networks, transportation systems, biology networks, and interdisciplinary research settings.

1. Introduction

Citation network analysis is essential for understanding how knowledge and ideas develop over time. Various analytical tools aim to reveal patterns, highlight important papers, and simplify complex relationships within these networks. While such tools can describe the *static* structure of a citation network, they often struggle to capture how communities and research areas evolve dynamically. Understanding these changes is crucial for identifying emerging trends, community splits or merges and the formation of new knowledge domains.

The goal of this project is to map structural changes in citation networks and to understand how groups of papers form communities, how research areas evolve, and when new domains emerge or when connections between existing areas shift.

The general approach is based on representing each paper as an embedding vector using GraphSAGE, which captures both local and global relationships within the network. K-Means clustering is then applied to group similar papers into clusters, and the quality of these clusters is evaluated using metrics such as the Silhouette Score to ensure that the clustering meaningfully reflects the network's structure.

Finally, changes in clusters over time are visualized using Alluvial Diagrams, illustrating the dynamics of communities and scientific domains.

2. Problem Formulation

Large weighted networks are widely used to represent complex systems such as scientific citation networks, and they often exhibit a modular organization into “communities” or clusters of nodes. When this structure changes across time, or between different realizations of the same system, it becomes difficult to determine which differences in network organization reflect genuine structural change and which are merely the result of random fluctuations in edge weights.

The central problem is to distinguish, within a sequence of large networks or within a single complex network, between clusters and large-scale structures that are statistically stable and robust, and those that could arise purely from noise, under the realistic constraint that at each time point we observe only one realization of the network and that nodes are not exchangeable.

This difficulty manifests in two main aspects:

1. Edge-weight noise – Random fluctuations in connection strengths can create or obscure visually apparent clusters without reflecting any real change in the underlying system.
2. Complexity of describing change – Even when structural changes do occur, summarizing and visualizing them clearly in large, highly modular networks over time is non-trivial.

3. Research Fields and Academic Context

This project belongs to the field of complex networks [7] and the study of knowledge and information dynamics. Its aim is to analyze dynamic citation networks, providing insight into how ideas, interests, and scientific information evolve over time.

Additionally, the project intersects several other scientific areas:

- **Information Science:** Analysis of network structure, identification of communities and detection of connections between papers, enabling a deeper understanding of the internal organization of knowledge.
- **Machine Learning and Data Science:** Use of numerical node representations (embeddings) and algorithmic methods to analyze and predict network structure, including dynamic clustering.
- **Social Sciences and Interdisciplinary Studies:** Analysis of knowledge dissemination, the evolution and expansion of research areas, and the identification of emerging scientific trends.

Overall, the project focuses on the dynamics and development of complex information systems, combining approaches from computer science, statistics, and social sciences to better understand how scientific knowledge evolves and adapts over time.

3.1 Citation Networks – Definitions and Structure

A network is defined as a mathematical structure consisting of nodes and edges that connect them. Many real-world networks are dynamic, meaning they change over time: new nodes are added, edges are created or removed, and relationships between nodes evolve. This enables analysis of how knowledge areas develop and how structures within the network change dynamically.

A citation network [8] is a specific type of dynamic network where each node represents a paper, and each directed edge indicates that one paper cites another. These networks reflect the flow of scientific information and allow the identification of connections and influences between different research fields.

The complexity of these networks, characterized by numerous connections and interactions among thousands or even millions of nodes, classifies them as complex networks, where knowledge domains, communities, and emerging research trends develop continuously.

Network analysis can be performed both at a local level, focusing on a node's relationships with its neighbors, and at a cluster level, examining the division of the network into communities or clusters, groups of nodes that frequently cite each other.

A community represents a research domain or shared scientific topic, and the connections between clusters provide insight into how ideas influence different areas. Tracking communities and clusters over time enables the study of network dynamics, including which communities remain stable, which split or merge, and which new domains emerge.

3.2 Embeddings and Clustering

In citation network analysis, each node is represented by an embedding vector, providing a numerical representation of the paper in a high-dimensional space. These embeddings capture both local connections with neighboring nodes and global relationships within the network, enabling mathematical processing and the application of machine learning algorithms.

The embedding vector defines the dimensions of this representation, with each dimension capturing a specific aspect of the network structure or the scientific context of the paper.

Primary clustering is performed using K-Means [5], which groups similar papers into homogeneous clusters representing meaningful communities within the network. During this process, uncertainty may arise, mainly due to noise or data inaccuracies, which can cause some papers to be assigned to inappropriate clusters.

The quality of clustering is evaluated using the Silhouette Score [10], which measures the separation between clusters and internal cohesion. A high score indicates clear and reliable clustering, while a low score suggests issues with cluster structure or the presence of noise affecting the clustering results.

3.2.1. Embedding Methods – General and Modern Approaches

Embedding methods are a key tool in analyzing complex networks. Their goal is to convert nodes into a high-dimensional numerical space, allowing mathematical processing and the application of machine learning algorithms. In classical approaches, embeddings are typically based on a node's local network structure and its direct connections with neighbors, while preserving similarity between nodes that share similar connection patterns.

In recent decades, modern approaches have emerged based on graph learning, particularly the GraphSAGE algorithm (Graph Sample and Aggregate) [1]. This method enables models to learn dynamic numerical representations of nodes by incorporating both local and global network information, combining data from neighboring nodes along with additional node features.

The main advantage of this method is its ability to handle large and dynamic networks, updating embeddings incrementally as new nodes and edges are added. Integrating these modern approaches enables more advanced analysis of citation networks, more accurate identification of communities and clusters, and a deeper understanding of the dynamics and evolution of knowledge domains[9] over time.

3.2.2 GraphSAGE Algorithm

GraphSAGE (Graph Sample and Aggregate) is a method for generating vector representations (embeddings) for nodes in large graphs. Unlike traditional embedding methods that require the entire graph during training, GraphSAGE learns general aggregation functions that allow it to generate embeddings even for nodes that were not observed during training. This makes it particularly suitable for large-scale and dynamic networks such as citation graphs.

Formally, given a graph $G = (V, E)$, each node has an initial feature vector $h_v^{(0)} = x_v \in \mathbb{R}^d$.

At each layer of the algorithm, GraphSAGE performs two key operations:

1. Sampling:

A fixed number of neighbors is sampled for each node to keep computation efficient even when nodes have very large neighborhoods.

2. Aggregation:

Features of sampled neighbors are aggregated using a learned function (e.g., mean or pooling). This aggregation $a_v^{(k)}$ summarizes local structural information,

and captures both structural and semantic information from the node's immediate neighborhood, summarizing local connectivity patterns in a compact form useful for downstream tasks.

The aggregated vector is then combined with the node's current representation and passed through a non-linear neural layer:

$$(1) \ h_v^{(k)} = \sigma(W^{(k)} \cdot \text{CONCAT}(h_v^{(k-1)}, a_v^{(k)}))$$

By stacking multiple layers, the algorithm extends the context beyond immediate neighbors, learning embeddings that capture broader structural patterns in the graph.

The final output is a low-dimensional embedding $h_v^{(K)}$ for each node v reflecting: the node's original features $h_v^{(0)} = x_v \in \mathbb{R}^d$, the aggregated information from its sampled neighbors $a_v^{(k)}$, and structural information from the graph up to K layers deep.

By leveraging this approach, GraphSAGE provides an efficient and scalable way to compute embeddings and represent nodes in large and dynamic networks, capturing both local and global network properties while enabling tasks such as clustering, community detection, and link prediction.

3.2.3 K-means Algorithm

K-means is a standard unsupervised clustering algorithm that partitions data into a fixed number (K) of groups (clusters) by minimizing the variation within each cluster. In graph analysis, it is typically applied to node embeddings (e.g., those produced by GraphSAGE), grouping nodes with similar representations in the embedding space. Even though K-means does not directly exploit the graph's edge structure, it is still widely used as a final clustering stage because it is conceptually simple, computationally efficient, and scales well to large datasets.

The algorithm operates iteratively through two main steps:

Assignment: Each embedding $z_i \in \mathbb{R}^d$ is assigned to the nearest centroid based on Euclidean distance.

Update: Each centroid is recomputed as the mean of the points currently assigned to its cluster.

These steps are repeated until assignments stabilize or a maximum number of iterations is reached.

Inputs

- Node embeddings: $Z = \{z_1, z_2, \dots, z_n\}$, where $z_i \in \mathbb{R}^d$
- The desired number of clusters: $K \in \mathbb{N}$

Output

- A cluster label for each node: $c_i \in \{1, \dots, K\}$
- The final centroids: $\{\mu_1, \mu_2, \dots, \mu_K\}$

3.2.4 Silhouette Score

The Silhouette Score is a metric used to evaluate the quality of clustering results by measuring how well each data point fits within its assigned cluster compared to other clusters. It provides a clear indication of both cohesion within clusters and separation between clusters, helping to determine whether the clustering structure is meaningful and reliable.

For each node i in the dataset, the Silhouette Score is calculated based on two key measurements:

Cohesion (a)

The average distance a_i between node i and all other points within the same cluster C_i :

$$(2) \ a_i = \frac{1}{|C_i| - 1} \sum_{j \in C_i, i \neq j} d(i, j)$$

Separation (b)

The average distance b_i between node i and all points in the nearest neighboring cluster.

Higher values indicate clearer separation from other clusters. The Silhouette Score for a single node i is:

$$(3) \ s_i = \frac{b_i - a_i}{\max(a_i, b_i)}, \quad s_i \in [-1, 1]$$

- A score close to 1 indicates correct and well-separated cluster assignment.
- A score around 0 indicates boundary points.
- A score close to -1 indicates likely misclassification.

Over Silhouette Score

By averaging the Silhouette Scores of all nodes, a single metric for the entire clustering result is obtained:

$$(4) \quad S = \frac{1}{n} \sum_{i=1}^n s_i$$

A higher S reflects well-separated and internally cohesive clusters; lower score indicates overlapping clusters or potential clustering issues.

The Silhouette Score is widely used due to its simplicity, interpretability, and applicability across a variety of clustering algorithms, and it is also helpful for comparing different clustering configurations or selecting the optimal number of clusters.

3.2.5 Jaccard Similarity

Jaccard Similarity [3] is a widely used metric for measuring the similarity between two sets, defined as the ratio between the size of their intersection and the size of their union. It is particularly appropriate for binary membership representations, where elements either belong to a set or not, and provides a simple and interpretable measure of overlap.

Given two sets A and B , the Jaccard Similarity is defined as the number of shared elements divided by the total number of distinct elements across both sets:

The resulting Jaccard Similarity score ranges from 0 to 1, where values close to 1 indicate highly similarity, and values close to 0 indicate minimal or no overlap.

Due to its simplicity and direct interpretability, Jaccard Similarity is commonly employed to compare set-based structures and to assess the degree of overlap between different groupings [2].

3.2.6 Hungarian Method

The Hungarian algorithm [4] solves the assignment problem optimally by transforming the cost matrix so that the best assignments appear as zeros. It first

performs row and column reductions, then covers all zero entries with the minimum number of lines.

- If K lines are required, an optimal one-to-one assignment can be selected.
- If not, the matrix is adjusted to create additional zeros, and the process repeats.

This iterative procedure efficiently identifies the minimum-cost matching between two sets of equal size

3.2.7 Visualization – Alluvial Diagrams

Alluvial Diagrams are a visualization method used to illustrate cluster structures change over time. In citation networks, these diagrams show how groups of papers evolve, how clusters split, merge, appear, disappear or form new communities, and how nodes move between communities.

Alluvial Diagram consists of blocks representing clusters at specific time points and flows representing node transitions, with flow width indicating the number of nodes. This method highlights cluster dynamics, shifts between research areas, and provides an intuitive view of large, complex networks. They can display clustering results (e.g., from GraphSAGE + K-Means) across time, tracking the evolution of research areas and community relationships across multiple time periods.

4. Requirements

This section defines the main requirements of the proposed system for analyzing structural changes in dynamic citation networks. The requirements are divided into functional and non-functional requirements in order to clearly specify the expected system behavior and the quality constraints under which it is designed to operate.

4.1 Functional Requirements

ID	Requirement Description
FR1	The system shall accept a dynamic citation network as input, including nodes (papers), edges (citations), and associated temporal information.
FR2	The system shall partition the dynamic network into discrete temporal snapshots based on a user-defined time window.
FR3	The system shall generate node embeddings for each snapshot using a graph-based embedding method (GraphSAGE).

FR4	The system shall perform community detection independently for each snapshot using K-Means clustering.
FR5	The system shall automatically select the optimal number of clusters for each snapshot based on clustering quality metrics.
FR6	The system shall evaluate clustering quality using the Silhouette Score.
FR7	The system shall assess clustering robustness by executing multiple K-Means runs with different random seeds.
FR8	The system shall align clusters across different runs and consecutive temporal snapshots using the Hungarian Method.
FR9	The system shall quantify similarity between clusters using the Jaccard Similarity measure.
FR10	The system shall visualize the evolution of communities over time using Alluvial Diagrams.

Table.1 Functional requirements

4.2 Non-Functional Requirements

ID	Requirement Description
NFR1	Scalability: The system shall be capable of handling large-scale citation networks containing up to millions of nodes.
NFR2	Robustness: The system shall produce stable clustering results that are not highly sensitive to random initialization.
NFR3	Interpretability: The system output, including cluster assignments and visualizations, shall be interpretable by domain experts without requiring deep technical expertise.
NFR4	Modularity: The system architecture shall allow independent modification or replacement of embedding, clustering, and visualization components.
NFR5	Reproducibility: Experimental results shall be reproducible given the same input data, configuration parameters, and random seeds.
NFR6	Performance: The system shall complete analysis for each snapshot within a reasonable computational time relative to snapshot size.

Table.2 Non-functional requirements

5. Model Description

This section presents a framework for analyzing the temporal evolution of community structures in dynamic complex networks. The model is designed to identify, track, and visualize changes in clusters over time, with the goal of capturing the emergence, persistence, merging, and splitting of communities. In the context of citation networks, clusters are interpreted as research topics or scientific fields, under the assumption that densely connected groups of papers correspond to coherent areas of research. The model integrates several advanced algorithms to support accurate cluster detection, alignment between consecutive temporal snapshots, and intuitive visualization of the results.

5.1 Graph Input and Dynamic Network Representation

The input to the model is a dynamic graph $G = (V, E, T)$, where nodes V represent entities in the network (e.g., academic papers), edges E represent relationships between nodes (e.g., citation links), and T denotes temporal information associated with nodes and/or edges. The network evolves over time as new nodes and edges are added.

To enable temporal analysis, the dynamic graph is partitioned into a sequence of discrete temporal snapshots using a fixed time window Δt . Each snapshot captures the state of the network within a specific time interval. The choice of Δt reflects a trade-off between temporal resolution and structural stability: smaller windows allow finer-grained temporal analysis but may introduce noise, whereas larger windows emphasize long-term structural trends.

5.2 Node Representation via Graph Embeddings

For each temporal snapshot, node representations are learned using GraphSAGE, a neighborhood-based graph embedding method. GraphSAGE [1] generates a numerical vector embedding for each node by aggregating information from its local neighborhood, thereby encoding structural properties of the network within the given snapshot.

Embeddings are learned independently for each snapshot. As a result, the model does not enforce temporal consistency at the embedding level. Instead, it assumes that structural similarity within each snapshot is sufficient for identifying meaningful communities, while temporal alignment is handled at the cluster level in later stages of the framework. This design choice simplifies the modeling process while still allowing the analysis of community evolution over time.

5.3 Community Detection within Snapshots

Community detection is performed independently for each snapshot by applying K-Means clustering to the node embeddings. Since the number of communities is not known a priori and may vary over time, the model evaluates multiple values of K (e.g., from 2 to 15) within a predefined range. For each candidate K , the Silhouette Score is computed to assess clustering quality based on intra-cluster cohesion and inter-cluster separation in the embedding space. The value of K that maximizes the Silhouette Score is selected as the optimal number of clusters [6] for the given snapshot is selected. Allowing K to vary across snapshots enables the model to capture the emergence of new research fields, the disappearance of outdated topics, and changes in the granularity of scientific communities over time.

5.4 Robustness to Random Initialization

After selecting the optimal K for each snapshot based on the Silhouette Score, robustness to random initialization is assessed by executing K-Means [5] multiple times on the same snapshot using the selected K while varying the random seeds. The resulting clusterings are then aligned using an optimal assignment procedure using the Hungarian Method, and the agreement between corresponding clusters is quantified using the Jaccard Similarity, computed over the sets of nodes assigned to each cluster. High Jaccard values indicate consistent community assignments across different initializations, whereas low values suggest sensitivity to initialization and reduced clustering stability.

5.5 Temporal Alignment of Communities

To track the evolution of communities across time, clusters from consecutive snapshots are aligned using the Hungarian Method. This alignment step establishes correspondences between communities at time and time, enabling the identification of persistent, emerging, merging, and splitting communities. The alignment is based on a similarity measure between clusters, such as Jaccard similarity computed over the sets of nodes assigned to each cluster. This similarity measure is used to construct a cost matrix, which serves as input to the Hungarian algorithm. By maximizing overall similarity between matched clusters, the model provides a principled mechanism for tracing community evolution across snapshots.

5.6 Visualization of Community Evolution

Finally, the results of the temporal alignment are visualized using Alluvial Diagrams, which illustrate the flow of nodes between communities over time. These visualizations provide an intuitive representation of community stability and change, highlighting long-lived clusters as well as structural transitions such as community splits, merges, and the emergence of new fields.

This framework provides a clear and structured view of cluster dynamics over time, allowing the assessment of cluster stability and variability, and supporting the identification of meaningful trends and patterns within dynamic networks.

(The structural partitioning of the network into modules and the distinction between genuine changes and noise is illustrated in **Appendix B, Fig.2, Fig.3**)

5.7 Pseudocode

```
# INPUT:

# G      : Dynamic graph with nodes and edges

# DELTA_T    : Time interval for snapshots

# K_MIN, K_MAX : Min and max number of clusters

# RANDOM_SEED : Random seed for reproducibility

# user_defined_runs : Number of runs for robustness evaluation

# OUTPUT:

# snapshots : List of Temporal snapshots with:

#           - optimal clustering

#           - Jaccard robustness scores

#           - inter-snapshot cluster matching

# Alluvial visualization of cluster evolution

# Step 1 – Load Dynamic Graph

G = load_graph("network_data")

# Step 2 – Temporal Snapshot Partitioning

snapshots = create_snapshots(G, delta_t = DELTA_T)

# Step 3 & 4 – Embedding and Clustering per Snapshot

for each snapshot in snapshots:

    # Input : snapshot graph

    # Output: node embeddings

    embeddings = GraphSAGE(snapshot)

    best_score = -∞
```

```

best_k = None

best_partition = None

# Step 4 – Optimize number of clusters using Silhouette Score

for k in range(K_MIN, K_MAX + 1):

    # Input : embeddings, number of clusters k

    # Output: cluster assignments

    clusters = KMeans(embeddings,k = k,

                        random_state = RANDOM_SEED,n_init = 10)

    score = SilhouetteScore(embeddings, clusters)

    if score > best_score:

        best_score = score

        best_k = k

        best_partition = clusters

snapshot.clusters = best_partition

snapshot.optimal_k = best_k

# Step 5 – Clustering Robustness Evaluation

# Input : embeddings, best_k, num_runs

# Output: Jaccard robustness score

snapshot.jaccard_robustness = compute_jaccard_robustness(

    embeddings,k = best_k,

    num_runs = user_defined_runs,align_method = "HungarianMethod")

# Step 6 – Temporal Alignment of Clusters Across Snapshots

for i in range(1, length(snapshots)):

    prev_clusters = snapshots[i - 1].clusters

    curr_clusters = snapshots[i].clusters

    # Input : cluster assignments from snapshots t and t+1

    # Output: matched clusters across time

```

```
similarity_matrix = compute_jaccard_similarity(prev_clusters, curr_clusters)
```

```
cost_matrix = 1 - similarity_matrix
```

```
snapshots[i].cluster_matching = HungarianMethod(cost_matrix)
```

```
# Step 7 – Visualization
```

```
# Input : snapshots with clusters and matching
```

```
# Output: alluvial diagram
```

```
VisualizeAlluvial(snapshots)
```

5.8 Model Flowchart

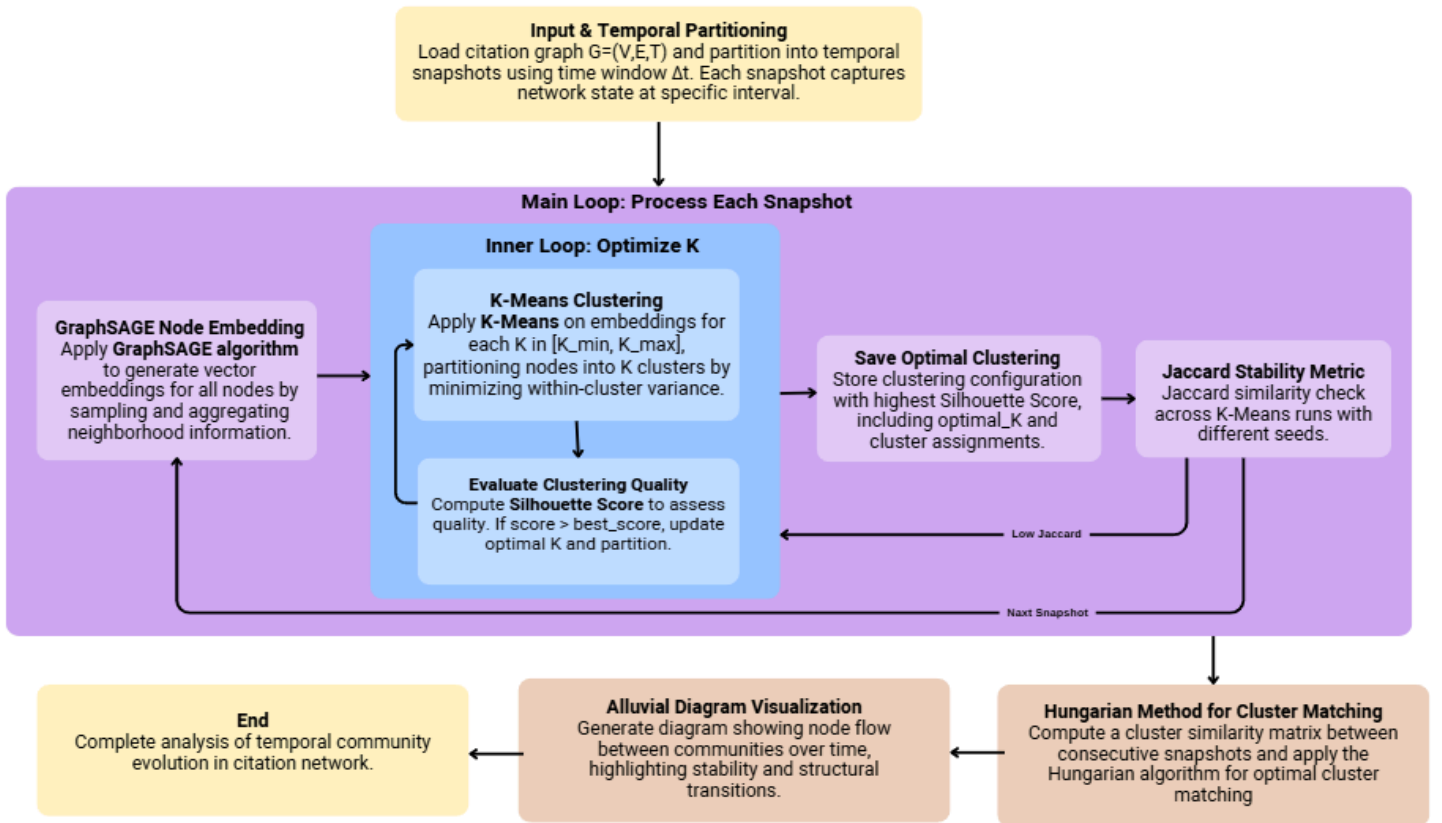


Fig.1 Dynamic citation network analysis algorithm

6. Statistical Analysis Methods and Implementation Technologies

This section presents the statistical methods that will be used to analyze the results of the proposed model, as well as the technological choices proposed for implementing Phase B of the project. These methods and tools are designed to ensure that the detected communities are meaningful, stable, and not artifacts of noise, random initialization, or arbitrary temporal segmentation.

6.1 Statistical Methods for Analyzing Model Results

To evaluate the quality, robustness, and temporal behavior of the detected communities, several complementary statistical and analytical measures will be applied.

6.1.1 Cluster Quality Evaluation

Clustering quality within each temporal snapshot will be primarily assessed using the Silhouette Score. This metric captures both **Intra-cluster cohesion**, reflecting the similarity of within the same cluster, and **Inter-cluster separation**, reflecting how well different clusters are separated in the embedding space. By computing the Silhouette Score [10] for multiple candidate values of K, the model selects the number of clusters that maximizes clustering quality for each snapshot. This adaptive selection allows the number of detected communities to vary over time, reflecting realistic changes in the structure of scientific domains.

6.1.2 Robustness to Random Initialization

To assess sensitivity to random initialization, clustering robustness will be evaluated using the Jaccard Similarity [3] as a consistency measure between clustering outcomes. In particular, the Jaccard Similarity will quantify the overlap between two sets and will be computed between the node memberships of corresponding clusters obtained from different K-Means initializations on the same snapshot.

Higher Jaccard values will indicate strong agreement between clustering solutions, suggesting that the identified communities reflect stable structures rather than initialization-dependent variability. Lower values will suggest increased sensitivity to random initialization.

6.1.3 Structural Change Analysis

Structural changes in the citation network can be analyzed through longitudinal examination of cluster-level properties, including cluster size distributions over time, the frequency and magnitude of cluster splits and mergers, and the appearance and disappearance of clusters. These quantitative measures can be later complemented by visual inspection using Alluvial Diagrams to support both statistical and intuitive interpretations of network evolution.

6.2 Implementation Technologies (Phase B)

The implementation in Phase B will rely on modern and widely adopted technologies for large-scale graph analysis, machine learning, and visualization.

6.2.1 Programming Language and Execution Environment

The system will be implemented in Python and will be executed in the Google Colab environment. Python will be selected due to its extensive ecosystem for graph analysis, machine learning, statistical evaluation, and visualization

6.2.2 Core Libraries and Frameworks

Category	Tool / Library	Purpose
Graph Embedding and Learning	PyTorch, PyTorch Geometric	Implementation of GraphSAGE and scalable graph learning methods for node embedding in dynamic citation networks
Clustering and Statistical Analysis	scikit-learn	K-Means clustering, Silhouette Score computation, and additional clustering evaluation metrics
Numerical Computation and Data Handling	NumPy, Pandas	Efficient numerical computation and structured data processing
Graph and Network Processing	NetworkX	Representation and analysis of graph structures, including node-level and cluster-level statistics
Optimization and Matching	SciPy	Implementation of the Hungarian Method for optimal alignment of clusters across time
Visualization	Plotly, Matplotlib	Generation of Alluvial-style diagrams and interactive visual analysis of community evolution

Table 3 . Implementation Technologies and Libraries

6.2.3 Visualization and Result Interpretation

Alluvial Diagrams will be used as the primary visualization tool to represent community evolution over time. These diagrams will visually highlight persistent research domains, cluster splits and mergers, and emerging and declining scientific topics. The visual outputs can be interpreted alongside the statistical metrics described above, enabling a comprehensive analysis that combines quantitative stability assessment with qualitative structural insights.

6.3 Anticipated Technical Challenges

The transition from theoretical design to practical implementation is expected to introduce several technical challenges. These challenges are anticipated based on the inherent complexity of dynamic graph processing, data availability constraints,

and the demands of effective visualization. The primary challenges are outlined below:

6.3.1 Scalability and Dynamic Graph Processing

Handling large-scale dynamic citation networks presents significant computational and memory-related challenges. As the network evolves over time, embedding generation and clustering must be performed repeatedly across multiple temporal snapshots, leading to increased computational overhead. Ensuring that these operations scale efficiently while maintaining acceptable performance and resource utilization constitutes a major technical challenge for the proposed system.

6.3.2 Identification of a Suitable Dataset

Selecting an appropriate dataset that satisfies the project's requirements is expected to be a non-trivial task. The dataset must represent a dynamic citation network with well-defined and reliable temporal information and include meaningful node attributes, such as topic, authorship, or publication venues. Additionally, the dataset should be sufficiently large to support robust analysis, public accessibility and conform with academic usage standards. These constraints may significantly limit the availability of suitable datasets.

6.3.3 Complexity of Visualization

Effectively visualizing the evolution of communities and topic transitions over time presents additional challenges. Dynamic networks require visualization techniques that preserve clarity, readability, and interpretability across multiple temporal snapshots. Specific difficulties include handling variable cluster sizes, preserving continuity in flow-based visualizations, and ensuring that visual representations remain informative under different data distributions.

7. Evaluation and Testing Plan

This phase focuses on verifying that the planned system for analyzing temporal changes in citation networks will function correctly, produce meaningful results, and achieve the project objectives. The suggested evaluation is designed to emphasize both quantitative accuracy and qualitative interpretability of the detected communities and their evolution over time.

7.1 Evaluation

The system's performance will be assessed using both real-world citation datasets and synthetic networks with known structural properties and temporal and transitions. This combined evaluation enables controlled validation alongside realistic scenario testing. The evaluation will be based on the following criteria:

Cluster Quality

Cluster quality will be assessed using the Silhouette Score, which will measure intra-cluster cohesion and inter-cluster separation for each temporal snapshot. This metric ensures that the detected clusters are compact and well-separated in the embedding space.

Temporal Stability

Temporal stability will be used to evaluate the persistence of clusters across consecutive snapshots. For each cluster C_t at time t , its similarity to the matched cluster C_{t+1} at time $t + 1$ will be computed using the Jaccard Similarity index:

$$(5) \quad J(C_j^t, C_j^{t+1}) = \frac{|C_j^t \cap C_j^{t+1}|}{|C_j^t \cup C_j^{t+1}|}$$

where the intersection represents the number of shared nodes between the two clusters, and the union represents the total number of distinct nodes appearing in either cluster.

The average similarity across all clusters can provide a stability score for each snapshot, as well as an overall stability measure for the network. High scores indicate persistent and well-defined research domains, while lower scores will suggest cluster splits, mergers, or noise-driven fluctuations.

Robustness to Random Initialization

To assess robustness under algorithmic variability, the K-Means clustering algorithm will be executed multiple times (R runs) with different random seeds. Similarity between the resulting clustering solutions will be measured to verify reproducibility. Significant variability across runs will indicate sensitivity to initialization and may suggest that the detected clusters do not represent stable topics in the citation dataset.

Structural Change Detection

Structural changes in the network will be analyzed through quantitative analysis of cluster splits, mergers, and the emergence or disappearance of clusters over time. This analysis will confirm whether the system accurately captures dynamic structural patterns in the citation network.

Comparisons will be conducted between the proposed framework and reference implementations or expected outcomes to validate accuracy and correctness, while also monitoring computational efficiency.

7.2 Testing Plan

The following table summarizes the planned testing procedure and maps evaluation objectives to concrete system components and test cases.

Index	Expected Result	Test Description	Component
1	Graph is preprocessed correctly (no self-citations, normalized weights)	Test graph preprocessing	CitationNetwork
2	Nodes contain accurate feature vectors	Test feature integration	CitationNetwork
3	Node embeddings generated for all nodes	Generate embeddings using GraphSAGE	GraphEmbeddingGenerator
4	Optimal cluster labels assigned to each node	Perform clustering on embedding vectors	ClusteringAlgorithm (K-Means)
5	Cluster assignments stable across multiple runs	Cluster Stability Test	ClusteringAlgorithm (K-Means)
6	Persistent, merged, split, and new clusters correctly identified	Compare clusters between consecutive snapshots	ClusterEvolutionTracker
7	Alluvial diagram representing cluster transitions is generated	Construct Alluvial diagram	AlluvialDiagramConstructor
8	Insights on trends (emerging, stable, declining clusters) produced	Interpret trends from clusters and alluvial diagram	InsightsInterpreter

Table 4. Verification / Testing Matrix

This structured evaluation and testing procedure is designed to ensure that the project's objectives will be verified quantitatively, and that the resulting insights into citation network dynamics are meaningful, interpretable, and reproducible.

8. AI Tools and Prompts Used

During the research design and development phases of this project, several AI tools were employed to support writing, content structuring, and visualization of ideas. These tools served as research and productivity assistants, helping to improve clarity, organization, and iterative refinement of the methodology. The main tools used include:

ChatGPT (OpenAI GPT-4 / GPT-5)

ChatGPT was used to generate explanations, summaries of algorithms, pseudocode, and to assist in refining English academic text.

Example prompt: *“Explain GraphSAGE embeddings and K-Means clustering for dynamic citation networks in simple English, including pseudocode.”*

Gemini AI

Gemini AI assisted in generating textual content, reviewing report structure, and suggesting improvements in clarity and readability.

Example prompt: *“Provide a concise summary of Alluvial diagrams and their use in visualizing community evolution over time.”*

Google Colab (AI-assisted visualization)

Google Colab supported the creation of interactive flowcharts and visual illustrations of the model’s workflow.

Example prompt: *“Create a flowchart illustrating the steps of the GraphSAGE + K-Means clustering process for dynamic citation networks.”*

All AI-generated outputs were treated as preliminary drafts or supporting material. All final content, algorithms, and visualizations were manually verified and adjusted by the project team to ensure correctness, reproducibility, and academic integrity.

9. References

- [1] Hamilton, W. L., Ying, R., & Leskovec, J. (2017). Inductive representation learning on large graphs. *arXiv preprint arXiv*. <https://arxiv.org/abs/1706.02216>
- [2] Hubert, L., & Arabie, P. (1985). Comparing partitions. *Journal of Classification*, 2(1), 193–218. <https://doi.org/10.1007/BF01908075>
- [3] Jaccard, P. (1901). Étude comparative de la distribution florale dans une portion des Alpes et des Jura. *Bulletin de la Société Vaudoise des Sciences Naturelles*, 37, 547–579. <https://www.biodiversitylibrary.org/page/4612088>
- [4] Kuhn, H. W. (1955). The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2), 83–97. <https://doi.org/10.1002/nav.3800020109>
- [5] Lloyd, S. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2), 129–137. <https://doi.org/10.1109/TIT.1982.1056489>
- [6] Milligan, G. W., & Cooper, M. C. (1986). A study of the comparability of external criteria for hierarchical cluster analysis. *Multivariate Behavioral Research*, 21(4), 441–458. https://doi.org/10.1207/s15327906mbr2104_5
- [7] Newman, M. (2018). *Networks*. Oxford University Press. <https://doi.org/10.1093/oso/9780198805090.001.0001>
- [8] Price, D. J. de S. (1965). Networks of scientific papers. *Science*, 149(3683), 510–515. <https://doi.org/10.1126/science.149.3683.510>
- [9] Rosvall, M., & Bergstrom, C. T. (2010). Mapping change in large networks. *PLoS ONE*, 5(1), e8694. <https://doi.org/10.1371/journal.pone.0008694>
- [10] Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, 53–65. [https://doi.org/10.1016/0377-0427\(87\)90125-7](https://doi.org/10.1016/0377-0427(87)90125-7)

10. Appendices

10.1 Appendix A – Full Algorithm Pseudocode

This appendix presents the full pseudocode of the core algorithms used in the proposed framework, including node embedding generation, clustering, robustness evaluation, cluster alignment, and visualization.

Algorithm 1 – GraphSAGE Embedding

- Input: $G(V,E,X)$: Graph with nodes V , edges E , and features X
- Output: Node embeddings $Z \in \mathbb{R}^{|V| \times d'}$
- Notes: Generates node embeddings through iterative neighborhood aggregation, ensuring reproducible and scalable representation learning.

Function GraphSAGE(G):

For each node v in V :

$z_v = X[v]$ # Initialize embedding with node feature

For hop $h = 1$ to H :

For each node v in V :

$N_v = \text{neighbors}(v)$

$h_v = \text{mean}(z_u \text{ for } u \text{ in } N_v)$

$z_v = \text{concat}(z_v, h_v)$

$z_v = \text{apply_non_linearity}(z_v)$

$Z = [z_1, z_2, \dots, z_{|V|}]^T$

Return Z

Algorithm 2 – K-Means Clustering

- Input: Z : Node embeddings; k : Number of clusters
- Output: C : cluster assignment labels

Function KMeans(Z, k):

Initialize k centroids randomly

Repeat until convergence:

For each point i in Z :

Assign i to nearest centroid

For each cluster j:

centroid_j = mean(points in cluster j)

Return cluster assignments C

Algorithm 3 – Silhouette Score

- Input: Z: Node embeddings, C: Cluster assignments
- Output: s -average silhouette score

Function SilhouetteScore(Z, C):

For each point i:

a_i = average distance to other points in same cluster

b_i = min(average distance to points in other clusters)

s_i = (b_i - a_i) / max(a_i, b_i)

s = mean(s_i for all i)

Return s

Algorithm 4 – Jaccard Robustness

- Input: Z: Node embeddings; k: Number of clusters; num_runs: Number of K-Means executions; alignment method: Cluster alignment method (e.g., Hungarian Method)
- Output: Average Jaccard similarity score

Function ComputeJaccardRobustness(Z, k, num_runs, align_method):

results = []

For run = 1 to num_runs:

clusters_run = KMeans(Z, k)

results.append(clusters_run)

similarities = []

For each pair (r1, r2) in results:

aligned = AlignClusters(r1, r2, method=align_method)

jaccard_score = ComputeJaccard(aligned)

```
similarities.append(jaccard_score)
```

```
Return mean(similarities)
```

Algorithm 5 – Hungarian Method

- Input: M: cost matrix representing inter-cluster dissimilarities
- Output: Optimal cluster assignment minimizing total cost

Function HungarianMethod(M):

```
assignment = SolveAssignment(M)
```

```
Return assignment
```

Algorithm 6 – Alluvial Diagram Visualization

- Input: Temporal snapshots with cluster labels
- Output: Visual representation of clusters evolution over time

Function VisualizeAlluvial(snapshots):

```
For each snapshot t:
```

```
    Draw clusters as blocks
```

```
For each consecutive pair (t, t+1):
```

```
    Connect blocks according to cluster matching
```

```
Render diagram with color-coded clusters
```

10.2 Appendix B – Visualizations

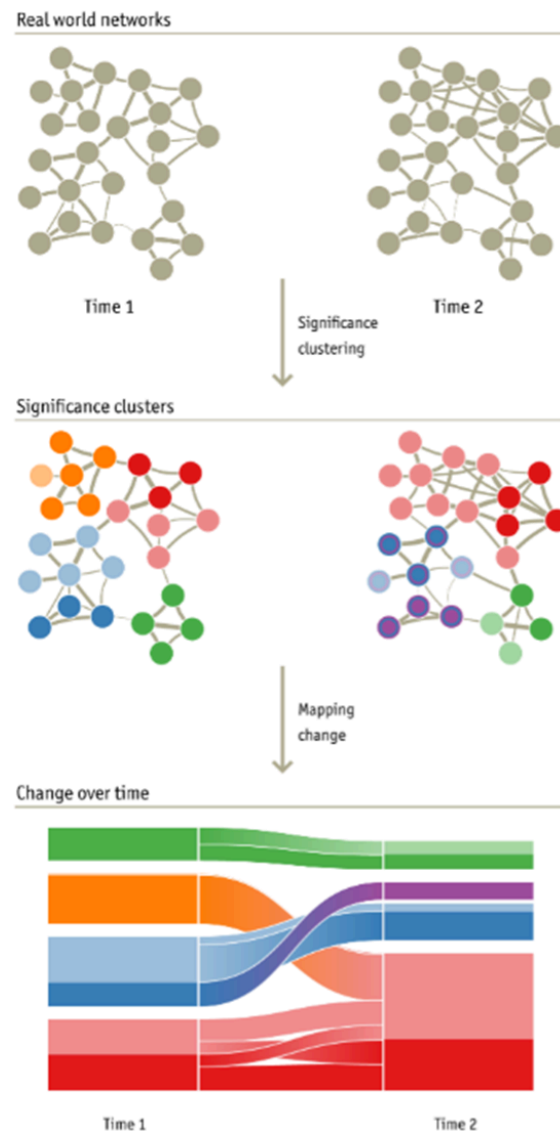


Figure 2: Structural analysis of community evolution in a dynamic network

Fig 2. Is an illustrative example demonstrating how a complex network can be partitioned into statistically significant modules, enabling the differentiation between genuine structural changes and random noise in the network dynamics [9]. The figure consists of three subfigures:

- (a) Real-world network snapshots at two consecutive time steps, representing the raw network structure prior to analysis.
- (b) Identification of statistically significant communities within each snapshot through clustering, highlighting the modular organization of the network at each time step.
- (c) Temporal mapping of communities using an Alluvial diagram, illustrating cluster evolution, including persistence, splits, and merges between Time 1 and Time 2.

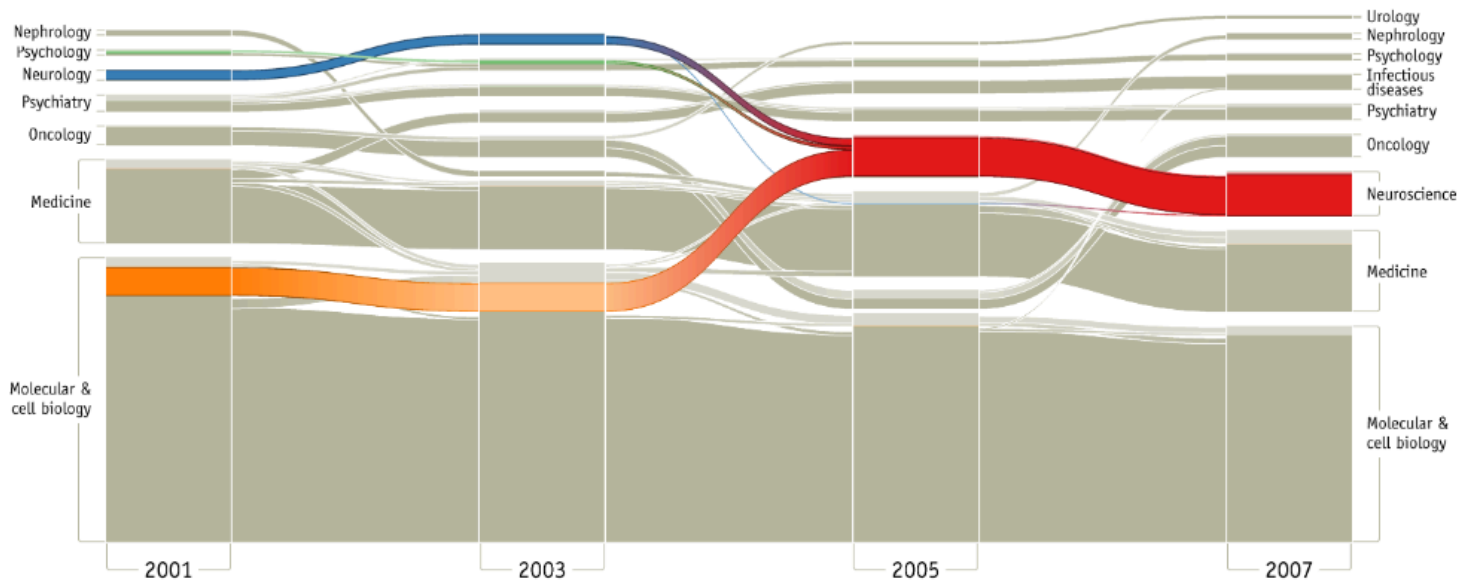


Figure 3: Illustrative Alluvial Diagram of Community Evolution.

Fig.3 illustrates the representation of structural changes in a dynamic network, highlighting how communities merge, split, and persist over time.